

Using the cloud to build a computing lab

Samuel Alves, Paula Prata
Dep. de Informática da Universidade da Beira Interior
Instituto de Telecomunicações
Covilhã, Portugal
samuel91a@hotmail.com, pprata@di.ubi.pt

Abstract — The management of a laboratory for the many classes/areas in an education institution makes a set of challenges for the IT department which face a growing pressure to provide the necessary computing resources at the lowest price. This paper proposes a platform named CSCLab for the provisioning and management of laboratories with the computing resources necessary to teach a class using the cloud. The system was designed to be used by non-experts in cloud technologies and implemented to run in OpenStack but it can be easily adapted to other platforms with similar services.

Keywords: *cloud; virtual laboratory; openstack; education; CSCLab;*

I. INTRODUCTION

In education institutions like universities and secondary schools the computing laboratories need to be pre-installed i.e. all software and hardware must be prepared in advance resulting in high costs. This presents the problem of content limitations by the a priori available software as well [1].

In order to make the necessary computing resources available, IT directs its attention to the cloud. The cloud is defined by National Institute of Standards and Technology – NIST [2] as a model that provides ubiquitous, convenient on-demand network access to a pool of resources that can be rapidly provisioned and released with a minimal management effort or interaction with the service provider. The cloud is composed of a set of services, defined by NIST as Software as a Service (SaaS), Platform as a Service (PaaS) and Infrastructure as a Service (IaaS). These models of services are used to separate the kind of access and resources provided to the user, SaaS model provides the user with access to applications deployed in the cloud, PaaS model allows users to deploy their applications to the cloud using specific programming languages and IaaS model, being the less restrictive model, provides the users with the ability to dispose of the basic computing resources (processing, storage, network) in order to run arbitrary software. There are some models of deployment for a cloud platform: (a) Public cloud; (b) Private cloud; (c) Hybrid cloud; (d) Federation of clouds. The public cloud infrastructure is provisioned for open use by the general public and the private cloud infrastructure is provided for exclusive use by a single organization comprising multiple consumers. The main difference between a public and a private cloud is based on proprietary terms and users' access rights. The hybrid model is a composition of distinct infrastructures (private and public). A Federation of clouds

works as a network of inter-connected clouds of two or more providers.

The cost of the public clouds is generally lower than those of private clouds, but their costs are constantly changing due to their pay-per-use system. With the increasing use of the cloud, private clouds become the elected model, as opposed to the public clouds, as their price remains constant [1].

Using a hybrid model, organizations can expand their capabilities by leasing resources from public clouds at reasonable costs [3]. In order to be able to migrate applications between clouds or just to benefit from special facilities of particular clouds, applications need to be portable to some level and technologically independent, being this a critical point due to the closed nature of the standards from public cloud providers. The problem with the federation of clouds is that it needs an agreement between cloud providers, which is hard to achieve, due to their closed nature. In this case, vendor-agnostic mechanisms are needed for the brokerage of cloud services [4]. A multi-cloud management platform is a platform that provides us with the ability to manage a set of clouds using a single platform and a single API to take control over every cloud registered in the multi-cloud platform. The multi-cloud approach is used to avoid single cloud provider lock-in [5].

We propose CSCLab which stands for Computer Science Cloud Laboratory, a platform for provisioning virtual laboratories with the necessary resources to teach a class. We started by deploying an IaaS OpenStack private cloud over which the platform is based on. In this article, we describe the requirements analysis where we identified the user types and functionalities, and then we present the data model used to manage and correlate the information from both the cloud infrastructure and our application. Then the most important implementations issues are presented. Although we are using a private cloud, the platform could be used with other clouds based on OpenStack. We have started some research on how to implement CSCLab in a multi-cloud environment and that preliminary study is also presented.

Our application must be as fluid as possible, so that even a person without the necessary knowledge to manage a cloud system is able to get a laboratory up and running and the application must be accessible even from a mobile platform.

In the next section we present a study of the related work followed by section III where we describe the system analysis for the proposed platform. Section IV describes the implementation of the CSCLab and in Section V the conclusion is presented.

This work was partly supported by Fundação para a Ciência e Tecnologia (FCT) under the project PEst-OE/EEI/LA0008/2013.

II. RELATED WORK

Virtual computing laboratories are widely used in universities to support teaching and research in areas such as engineering, computer sciences and information assurance [1] [6-8]. These virtual computing laboratories are typically provided through basic desktop virtualization, using services like Parallels [9] and Microsoft Hyper-V [10]. There are also some products that offer a complete set of tools (such as, a portal, a library and a deployment manager) for virtual labs like the VMWare Suite [11] (e.g. VMware ESX, Lab Manager and vCloud Director) and Apache VCL [12].

Apache VCL was developed at the University of North Carolina as part of a project to develop a virtual laboratory solution and then donated to Apache Foundation in 2008, and though being open source, it is based on software and hardware from IBM [6].

Infrastructure cloud providers like Rackspace [13], Amazon EC2 [14] and Windows Azure [15] provide us with the tools necessary to create and manage as many environments as we need. Also, providers like Amazon present us with solutions for the education community as well [16]. The cloud is also being used to provide storage for researchers [17]. But the interface presented by these providers is not the ideal to create virtual computing laboratories. There is a need for making the generation of laboratories easy even to non-experts.

There are some projects in the cloud that are aimed at researchers like the European Grid Infrastructure (EGI) [18] and FutureGrid [19]. EGI is a federation of infrastructure clouds and it is made up of numerous providers (European Intergovernmental Research Organizations, Organizations through Memoranda of Understanding and other organizations with strong collaborations with EGI) and FutureGrid provides a test-bed to conduct experiments by submitting experiment plans through which researchers can tackle complex investigation challenges.

Our platform emerges as a solution to avoid proprietary lock in's from product vendors like VMware and as an alternative from Apache VCL that though being open source it is based on software from IBM. Also, solutions that implement platforms like VMWare require a learning curve which may imply training tutorials and time to learn how to correctly use the services from the platform [7].

III. SYSTEM ANALYSIS

In order to provide our application with the necessary functionalities to make a work-friendly laboratory we first started by defining a set of requirements that make it possible to teach a class. In this section we describe the system requirements, the top level architecture and data model for the CSQLab.

A. Requirements analysis

The laboratory is a session in the cloud which has an authentication system, a set of machines, a networking system and possibly a storage system. A machine in the lab is a virtual machine based on a pre-existent image or a specifically created image. The virtual machine is also known as an instance of the image.

The platform must have a set of pre-built images with specific operating systems and software for different classes as well as base images (basically just the operating system). There must be a software repository with the software used in the institution.

Our platform has three user types by default which will have different levels of access to its resources and have some different functions associated with them.

The user type "administrator" has access to all the resources of the cloud and also the responsibility to manage it. The administrator has control over other users and their permissions.

The user type "lab manager" is responsible for one or more lab classes. The lab manager has access to the lab resources to create and manage them.

Finally, we have the user type "student" who is the user with fewer privileges and that may access to a virtual machine in the lab, can access the software repository and may create and manage a single volume of persistent data.

The students must be registered in the cloud in order to use the platform. They may enroll in one or more class labs and when the laboratory is created, the lab manager must confirm their enrollment.

Our platform implements the following set of functionalities:

- The lab manager may access and manage the software repository.
- The lab manager may create persistent laboratories.
- The lab manager may create one time laboratories.
- Inside a laboratory, the lab manager may create virtual machines/instances with specific images and flavors (a category of resources to be associated with a virtual machine e.g. ram, cpu, volume size).
- The lab manager may add new resources to a virtual machine and save it as a new image.
- The lab manager must be able to schedule the launching of a laboratory in order to avoid waiting for the machines to be ready for work.
- A lab manager must be able to see and get access to the student's machines.
- The lab manager can select a set of features about which he/she wishes to get informed when the student makes something in the virtual machine from the laboratory and may define a set of rules to be applied to the machines of the laboratory.
- A student must be able to use resources from only a single laboratory at a time.

- A student must be able to create and manage a volume to store persistent data.
- A student must be able to create a virtual machine when not using a laboratory to get access to his persistent volume.
- A student may create a virtual machine to use software from the repository.
- Only the administrator may delete software, virtual machine images and laboratories from the cloud.

B. Architecture

Our platform – CSCLab is made up of a private cloud, a database and a web application as can be seen in “Fig. 1”.

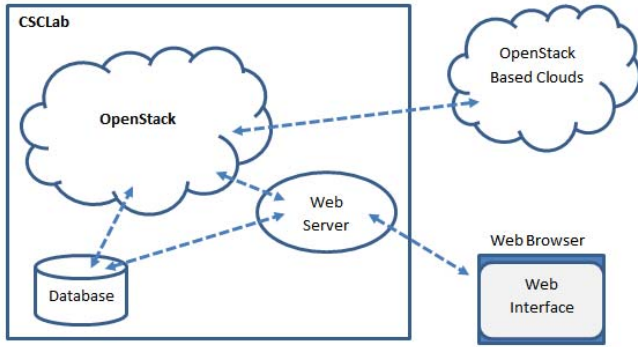


Figure 1 – CSCLab architecture

If the private cloud does not have the necessary resources to set up a laboratory, then CSCLab will use resources available through other clouds which are identified in our private cloud services. The cloud provides the resources for the set up of the

laboratories. The web application manages the interaction between the components of the platform and provides the users with an interface that can be accessed through a web browser.

C. Data Model

We created the data model presented in “Fig. 2” in order to describe the schematics for the relationships between the different entities of the platform. These entities represent the data which are necessary to create the functionalities of the platform.

In order to represent the relationships between the different entities, we use Unified Modelling Language – UML [20] cardinality notation. In the data model the entities we have created and the main entities from the cloud database which contain the information we need are presented. The entities have as a prefix the name of the service they belong to. We could duplicate this information to entities in our database but this is not a necessary step as we have access to this information whether through direct access to the cloud services database or through the cloud services endpoints.

The Laboratory entity represents the laboratories created in the platform. Each laboratory is composed of a unique identifier, its creation time, the identification of the user who created it, a number of virtual machines to be created by default, an extra field to save pertinent information about it and the image identification that will be used by each virtual machine of the laboratory. The Image entity has as attributes its id, the operating system and class to be taught. The attributes of the Class entity are the id, the name and a field for extra data. The Image and Class entities have a many-to-many relationship represented by the ImageClass table.

The Software entity contains the data related to the software available in the institution, namely the location url

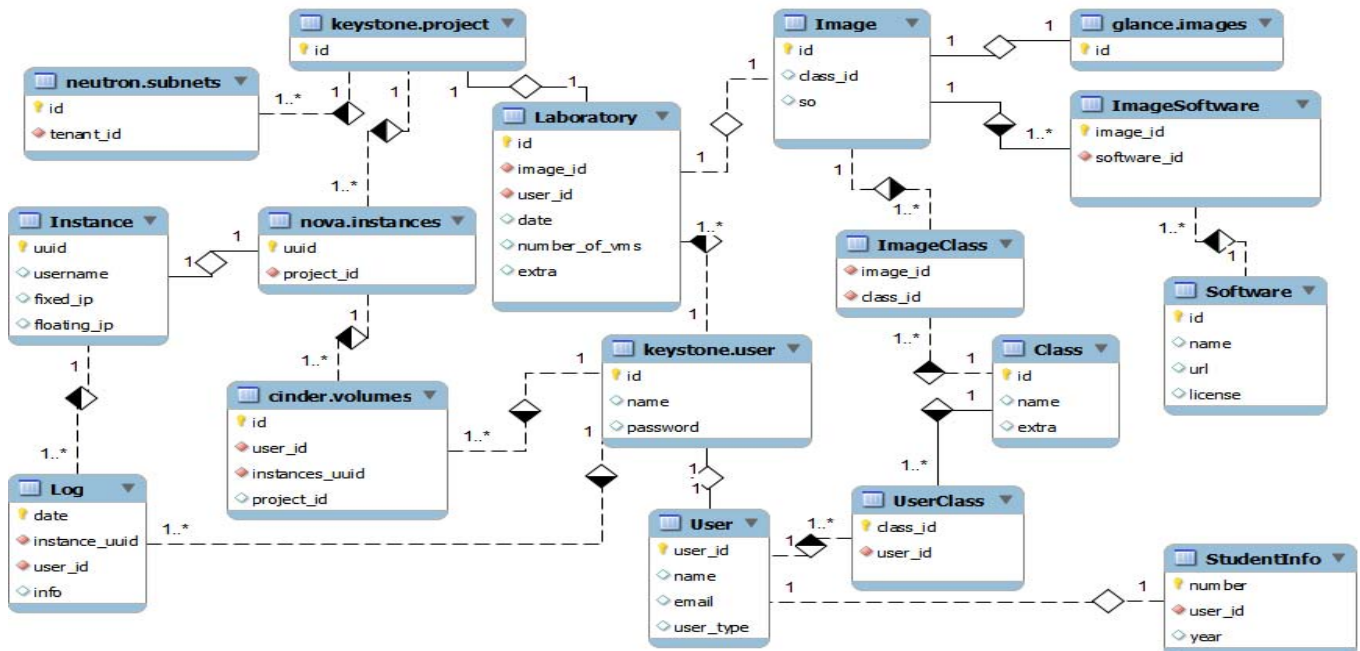


Figure 2 – CSCLab data model

and license. The Software and Image entities have a many-to-many relationship represented by the ImageSoftware table.

The User entity contains information about the users of the platform. Each user has its id, a name, an email and belongs to a specific user type. The User and Class entities have a many-to-many relationship represented by the UserClass table. If a user is a student, then additional information is created in the StudentInfo entity, which holds the student number and the year he/she attends.

The remaining entities will be described in the next section after presenting the services from the private cloud.

IV. THE CSCLAB PLATFORM

Our platform as seen in “Fig. 3” is the product of two main subsystems, i.e., the OpenStack Havana private cloud [21] which will provide us with the necessary computing resources and the web application which is composed of a MySQL database and an Apache web server. The user is able to get access to the web application through a web interface accessible from a web browser.

A. Private cloud

We implemented an OpenStack private cloud which provides us with access to the base components of computation. Our private cloud runs the minimum services necessary to test the functionalities that we must implement in our platform (see “Tab. 1”).

TABLE I. CLOUD PLATFORM SERVICES

Controller Node	Compute Node
Identity Service (Keystone) Image Service (Glance) Compute Service (Nova) Block Storage Service (Cinder) Network Service (Neutron)	Compute Service (Nova) Block Storage Service (Cinder) Network Service (Neutron)

The private cloud consists of two nodes (i.e., the controller node and a compute node) connected to a router, connecting the platform to the institution building and to the Internet. We chose to use Ubuntu 13.10 as the host system for our nodes, under which we installed the necessary OpenStack services.

The Image service named Glance, which runs in the controller node, contains all the information regarding the virtual machine images.

The Nova, Cinder and Network services are subdivided in controller services and compute services. Nova is the compute service from OpenStack and runs the API and metadata services on the controller side. On the compute node side it is where the virtual machines are created.

Cinder manages the storage system and Neutron manages the networks in the cloud.

The controller node runs the identity service (Keystone) which is used to register the users of the cloud as well as the services used by the cloud (e.g. Glance, Nova, Cinder and

Neutron) and uses it as the identity service for access to other clouds as well.

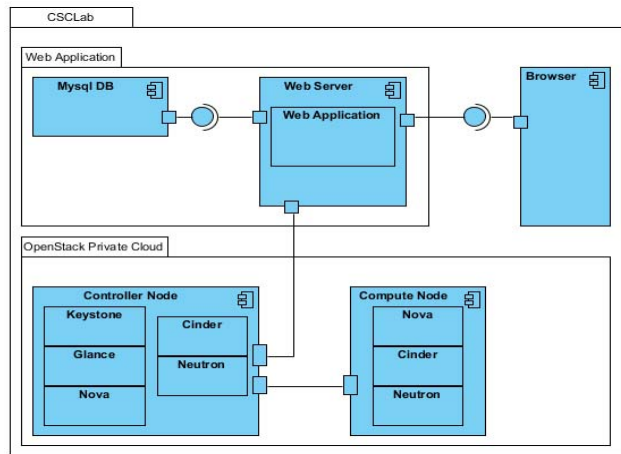


Figure 3 – System components

Each OpenStack service has its own database from which we use the following entities: glance.images, from which we get the id that uniquely identifies each image; keystone.project and keystone.user which are used to identify a set of resources from each of the laboratories and identifies the users of the platform; neutron.subnets is used to get information about the networks of each laboratory; nova.instances is used to identify each virtual machine from the laboratories; cinder.volumes is used to get information about the persistent volumes assigned to each user.

The main services were configured following an install guide from the documentation of Openstack with some adaptations in order to take advantage of the available hardware.

We decided to incorporate the network node into the controller node for testing purposes and the cinder volume service is running under the compute node instead of a dedicated node.

B. Web Application

Our web application subsystem is composed of a MySQL database, a web interface in HTML5 and a controller model. We opted to follow a Model-view-controller (MVC) architecture using the PHP Framework named CodeIgniter [22] because this way we separate the different layers of the web platform, thus allowing us to work just the layers we need without modifying the others.

By using HTML5 standards, in conjunction with CSS3 and using CSS media queries we provide users with access to the web interface of the application from any mobile device with a browser with HTML5 support. In order to access the services from the OpenStack cloud we will use the php-opencloud library [23] (i.e. PHP SDK for OpenStack powered clouds) which provides us with a more user-friendly way to make requests to the cloud using PHP.

Most of the services are fully implemented but just the web interface is still in a development stage.

C. System description

In our work we chose to implement an OpenStack based private cloud due to its open source nature which has put it under the spotlight. OpenStack is a solution that provides us with the ability to maintain our own IaaS private cloud.

All users must be registered in Keystone in order to get and manage resources from the platform. The unique identifier assigned to them is also used to distinguish the users in the entities we created to manage the additional information we need.

CSCLab platform makes it possible for a lab manager to choose the number of necessary machines, the operating system they run and the software set necessary for the class. It is possible to choose the type of resources every virtual machine (VM) will have (i.e., computing power, disk/volume size), the private networks necessary for communication between the VMs and the range of public IPs from a pool of public IPs available in order to get access to VMs in the network of the building. Every machine must have its own personalized login based in users' credentials.

Our platform provides the lab manager with the possibility to directly use one of the pre-defined images from our repository and personalize it by installing new software or any other personalization supported by the operating system of the image. The lab manager can then save the image state in the platform, creating a new image which better corresponds to its needs. This new image can then be used in a laboratory. To achieve this, we use the snapshot functionalities provided by the OpenStack cloud.

A laboratory instance is composed of a specific image, a given number of virtual machines and the information about its creation (e.g. date, time and other relevant data). In order to integrate it with the OpenStack private cloud we also need to create a container for the computational resources of the laboratory. To achieve this we use Tenants, a tenant is a container used to group or isolate resources and/or identity objects, a Tenant is also known as a Project in some of the OpenStack services like Nova. This container is registered in the entities from the database of the identity service (Keystone). Since each project is identified by a universal unique identifier, and assuming that we will use a different project for each laboratory, we use this identifier to uniquely identify each laboratory. This information will be an instance of Laboratory entity.

We chose to use a network topology which provides us with a per-tenant router with private networks in order for a separation between the networks of different tenants. This way it is also possible to have overlapping sub-networks between the tenants because access to the network happens through Secure Network Address Translation – SNAT or floating IPs (i.e., public IPs). This allows the laboratories to have separated private networks.

To each instance of the laboratory entity we may assign one or more private networks that will be used for communication between the instances of the laboratory. We also assign a pool of floating IPs, which is a subnet from the external network, and will be used for communication between users and

instances using protocols like SSH, VNC and RDP – Remote Desktop Protocol.

The images used by the virtual machines from the laboratory are stored in the image service of the cloud platform which stores information of each of its images in Glance database. For each of the images we need to have information about which operating system is used, which class it was assigned to and a list of the software installed. For consistency purposes in our entities we use the universal unique identifier which uniquely identifies the images in the Glance database.

A laboratory may have zero, one or more virtual machines running which are managed by the computing services of the cloud platform which stores its information in Nova database. These virtual machines have a volatile memory, i.e., when the instance is terminated, the data inside its volume is lost [6].

We could preserve the state of each virtual machine from the laboratory, but in order to accomplish that, we would need to save each virtual machine image to a volume which conducts to an inefficient resource usage (i.e., instead of one image for a laboratory, we would need one image for each student that attends the class/laboratory). So, we opted to create a volume for each student that he/she can use to preserve his/her data. This volume can be attached to the virtual machine that the users are using and detached when the instance is terminated. We assume that the student may only be assigned to only one laboratory at a time, since a volume is a detachable device much like a USB hard drive, i.e., it can only be attached to one virtual machine at a time [7]. In order to attach the volume to the right virtual machine, we need to identify the project/laboratory to which we need to transfer the volume. To transfer the volume between projects, it is necessary to know in which project it is and emit a transfer order which will then be accepted in the laboratory where it is needed. This is possible through a relatively new functionality included in Cinder (cinder-transfer) which makes it possible for a volume to be transferred between tenants.

To each of the virtual machines created in the laboratory we need to configure the user credentials with the username and password of the user in order to grant access to it. Using “cloud-config” scripts we can pass metadata (user credentials and personal volume) to the virtual machines in order to personalize it according to the user. The login information for each virtual machine is stored in the Instance entity.

The application must also have a logging system that uses a monitoring tool to provide the lab manager with feedback about what each student is doing. To better control the way students use their resources, we created a logging entity which will log relevant and pre-defined actions. This relevant information can be pre-defined by the lab manager and it is based on some rules like the launching of certain programs, instance restarts and some others that may be considered of interest. The logging information is stored in the Log entity.

D. Integration with a multi-cloud platform

We conducted a study of the implementation of multi-cloud platforms by testing some of the existing solutions. There are already some providers which provide multi-cloud solutions from which we point out RightScale [24], Dell Cloud Manager

[25] and Scalr [26]. From the open source world there is CompatibleOne [27], mOSAIC [28] and Scalr (open source version).

We tested CompatibleOne and Scalr by deploying and testing them with our OpenStack private cloud. Following the existent documentation for the first solution, we were not able to get a fully functional multi-cloud environment. Regarding Scalr, we got it to work, but by testing it we got to the conclusion that by using such a tool we would end up losing some functionalities of our cloud, like the management of the volumes (transfer capabilities) and Neutron functionalities.

There is always the option to use inter-cloud libraries but the problem persists in some of the functionalities. On the one hand, the cloud technology is in evolution (some services end up not being included in the existent versions of the libraries) and on the other hand, many infrastructure providers have closed standards.

In our future work, we intend to make a study of the best way to implement a multi-cloud solution for CSCLab.

V. CONCLUSION

Although the main laboratories from education institutions are usually built using local computing resources, nowadays this approach is evolving towards a strategy of moving to cloud computing solutions. In this paper we presented CSCLab, a proposal for a platform to manage computer science laboratories by using cloud resources trying to take advantage from the elastic cloud model. Using the cloud, computing resources can be allocated or released according to a greater or lesser demand.

This platform is intended to be used by non-experts in cloud technology. A laboratory is a session in the cloud with a configurable set of machines, a networking system and possibly a storage system. We have described the system requirements, the architecture and the data model which uses some entities from the databases existent in the cloud services. Then, we discussed the main technical details which we faced in the development of the CSCLab platform.

In future research, the CSCLab will be extended to be a multi-cloud solution.

REFERENCES

- [1] M. Uehara, "Proposal for BYOD based Virtual PC Classroom", 16th International Conference on Network-Based Information Systems, pp.377-382, 2013, doi: 10.1109/NBiS.2013.60.
- [2] P. Mell and T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology, Special Publication 800-146, 2012.
- [3] R. Calheiros, R. Ranjan, A. Beloglazov, C. De Rose and R. Buyya, "CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms," Wiley Online Library, 24 August 2010.
- [4] D. Petcu, "How to build a Reliable mOSAIC of Multiple Cloud Services," EWDCC '12 Proceedings of the 1st European Workshop on Dependable Cloud Computing, 2012, doi: 10.1145/2365316.2365320.
- [5] P. Miller, "Sector Roadmap: Muticloud management in 2013", GIGAOM Research, September 2013.
- [6] S. D. Burd, X. Luo and A. Seazzu, "Cloud-Based Virtual Computing Laboratories", 46th Hawaii International Conference on System Sciences, pp. 5079-5088, 2013, <http://doi.ieeecomputersociety.org/10.1109/HICSS.2013.131>.
- [7] X. Wuang, G. Hembroff and R. Yedica, "Using VMware vCenter Lab Manager in Undergraduate Education for System Administration and Network Security," SIGITE'10, ACM conference on Information technology education, pp 43-52, 2010, doi: 10.1145/1867651.1867665.
- [8] L. Xu, D. Huang and W.-T. Tsai, "V-Lab: A Cloud-based Virtual Laboratory Platform for Hands-On Networking Courses," ITiCSE'12, 17th ACM annual conference on Innovation and technology in computer science education, pp.256-261, 2012, doi:10.1145/2325296.2325357.
- [9] Parallels, "<http://www.parallels.com/>", consulted on February 1th of 2014.
- [10] Microsoft Hyper-v, "<http://technet.microsoft.com/en-US/evalcenter/dn528863.aspx>," consulted on February 1th of 2014.
- [11] VMware, "<http://www.vmware.com/>," consulted on February 1th of 2014.
- [12] NC State University, "<http://vcl.ncsu.edu/papers-publications>", consulted on February 1th of 2014.
- [13] Rackspace, "<http://www.rackspace.com/>," consulted on February 1th of 2014.
- [14] Amazon EC2, "<http://aws.amazon.com/ec2/>," consulted on February 1th of 2014.
- [15] Windows Azure, "<http://www.windowsazure.com/en-us/>," consulted on February 1th of 2014.
- [16] Amazon, "<http://aws.amazon.com/education/>", consulted on February 1th of 2014.
- [17] A. Adshead, "University tiers its cloud system", CWeurope Magazine, page 15, January 2014.
- [18] European Grid Infrastructure, "www.egi.eu/infrastructure/index.html", consulted on February 1th of 2014.
- [19] Future Grid, "<https://portal.futuregrid.org/>", consulted on February 1th of 2014.
- [20] UML, "<http://www.uml.org/>", consulted on February 1th of 2014.
- [21] "OpenStack End User Guide - Havana," OpenStack, Feb. 5th of 2014.
- [22] CodeIgniter, "<http://ellislab.com/codeigniter>," consulted on February 1th of 2014.
- [23] Php-opencloud, <http://php-opencloud.com/>, consulted on February 1th of 2014.
- [24] Rightscale, "<http://www.rightscale.com/>", consulted on February 1th of 2014.
- [25] Dell Cloud Manager, "<https://www.enstratus.com/>", consulted on February 1th of 2014.
- [26] Scalr, "<http://www.scalr.com/>", consulted on February 1th of 2014.
- [27] CompatibleOne, "<http://www.compatibleone.org/bin/view/Main/>", consulted on February 1th of 2014.
- [28] mOSAIC, "<http://www.mosaic-cloud.eu/>", consulted on February 1th of 2014.